Docker & Kubernetes

Complete Interview Q&A Guide

100+ Questions with Detailed Answers

This guide covers all major Docker and Kubernetes interview topics with detailed, production-grade answers. Topics include core concepts, networking, storage, security, scaling, debugging, and advanced patterns.

🐳 DOCKER CORE CONCEPTS

Q1. What is Docker, and how does it differ from a Virtual Machine?

▶️ Answer:

Docker is an open-source platform that automates the deployment, scaling, and management of applications using containerization. It packages an application and all its dependencies into a lightweight, portable unit called a container.

Key differences from Virtual Machines:

- Architecture: VMs virtualize hardware via a hypervisor; Docker containers share the host OS kernel, running as isolated processes.
- Size: VM images are GBs (include full OS); Docker images are MBs (only app + dependencies).
- Startup time: VMs take minutes to boot; containers start in milliseconds.
- Isolation: VMs provide stronger hardware-level isolation; containers share the kernel (weaker but sufficient for most use cases).
- Portability: Docker images run identically on any system with Docker installed.
- Resource efficiency: Containers use less CPU/RAM because they don't need to emulate hardware.

Q2. Explain the Docker architecture.

▶️ Answer:

Docker uses a client-server architecture with three main components:

- Docker Client (docker CLI): The interface users interact with. Sends commands (build, pull, run) to the Docker Daemon via REST API.
- Docker Daemon (dockerd): Runs on the host machine. Manages Docker objects — images, containers, networks, volumes. Does the heavy lifting.

- Docker Registry: Stores Docker images. Docker Hub is the default public registry. Private registries (ECR, GCR, Harbor) are also supported.
- Docker Objects: Images (read-only templates), Containers (running instances of images), Volumes, Networks.

Q3. What is the difference between a Docker Image and a Docker Container?

▶️ Answer:

A Docker Image is a read-only, immutable blueprint/template used to create containers. It contains the application code, runtime, libraries, environment variables, and configuration. Images are built from a Dockerfile.

A Docker Container is a running (or stopped) instance of an image. It is writable, isolated, and ephemeral. Multiple containers can be created from the same image, each running independently.

Analogy: An image is like a class in OOP; a container is like an object (instance) of that class.

Q4. What is a Dockerfile? Walk me through writing one.

▶️ Answer:

A Dockerfile is a text file containing a series of instructions that Docker reads to automatically build an image. Each instruction creates a new layer. Example:

```
# Use official Node.js base image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files first (for layer caching)
COPY package*.json ./

# Install dependencies
RUN npm install --production

# Copy application source code
COPY . .

# Expose the port
EXPOSE 3000

# Define the startup command
CMD ["node", "server.js"]
```

Q5. What is the difference between CMD and ENTRYPOINT?

▶️ Answer:

Both define what command runs when a container starts, but they behave differently:

CMD: Provides default arguments that can be overridden from the command line. If you run 'docker run myimage bash', the CMD is replaced with 'bash'.

ENTRYPOINT: Defines the main executable that always runs. Arguments passed via docker run are appended to ENTRYPOINT, not replaced.

Best practice: Use ENTRYPOINT for the main executable and CMD for default arguments that can be overridden.

```
# CMD — can be fully overridden
CMD ["python", "app.py"]

# ENTRYPOINT — always executes; CMD becomes default args
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker run myimage test.py  → runs: python test.py
```

Q6. What is the difference between COPY and ADD in a Dockerfile?

▶️ Answer:

Both copy files from host to the container image, but ADD has extra features:

- COPY: Simply copies files/directories from the host to the image. Preferred for most use cases because it's explicit and transparent.
- ADD: Does everything COPY does, PLUS it can extract .tar archives automatically and fetch files from URLs.
- Best practice: Always prefer COPY unless you specifically need the tar-extraction or URL-fetching features of ADD.

Q7. What are Docker layers and why do they matter for caching?

▶️ Answer:

Each instruction in a Dockerfile (FROM, RUN, COPY, etc.) creates a new read-only layer. Layers are stacked on top of each other to form the final image.

Caching benefit: Docker caches each layer. If a layer hasn't changed, Docker reuses the cached version, making rebuilds much faster.

Key principle: Order instructions from least-changing to most-changing. Copy dependency files (package.json) before source code so that the dependency install layer is cached when only source code changes.

Q8. What is a multi-stage build and why would you use it?

▶️ Answer:

Multi-stage builds allow you to use multiple FROM instructions in one Dockerfile. Each FROM starts a new build stage. You can selectively copy artifacts from one stage to another, leaving behind everything you don't need.

Why use it: Drastically reduces final image size by keeping build tools out of the production image.

```
# Stage 1: Build
FROM golang:1.21 AS builder
WORKDIR /app
```

```
COPY . .
RUN go build -o myapp .

# Stage 2: Production (tiny image)
FROM alpine:latest
COPY --from=builder /app/myapp /usr/local/bin/
CMD ["myapp"]
```

🌐 DOCKER NETWORKING & STORAGE

Q9. What are the Docker networking modes?

▶️ Answer:

Docker provides several network drivers:

- bridge (default): Containers get their own IP on a virtual network. They can communicate via container name. Best for standalone containers on a single host.
- host: Container shares the host's network stack. No network isolation. Best for performance-critical apps.
- none: No networking. Container is completely isolated from the network.
- overlay: Spans multiple Docker hosts. Used in Docker Swarm for multi-host communication.
- macvlan: Assigns a MAC address to the container, making it appear as a physical device on the network.
- ipvlan: Similar to macvlan but at layer 3.

Q10. What is the difference between a Docker Volume and a Bind Mount?

▶️ Answer:

Both are used to persist data beyond a container's lifecycle, but they differ:

- Docker Volume: Managed by Docker, stored in /var/lib/docker/volumes/. Portable, easy to back up, can be shared between containers. Preferred for production.
- Bind Mount: Maps a specific path on the host filesystem into the container. Useful during development for live code reloading. Less portable as it depends on the host's directory structure.
- tmpfs mount: Stored in the host's memory only. Never written to disk. Useful for sensitive data.

Q11. How do containers communicate with each other on the same host?

▶️ Answer:

On the default bridge network, containers communicate via IP address. On a user-defined bridge network, they can also communicate via container name (DNS resolution).

Best practice: Always create user-defined bridge networks rather than using the default bridge, as they support automatic DNS resolution by container name.

```
docker network create mynet
```

```
docker run -d --name db --network mynet postgres
docker run -d --name app --network mynet myapp
# 'app' can reach 'db' at hostname 'db'
```

⚙️ DOCKER OPERATIONS & BEST PRACTICES

Q12. How do you reduce Docker image size?

▶️ Answer:

Several strategies to reduce image size:

- Use a minimal base image: alpine (~5MB) instead of ubuntu (~70MB).
- Multi-stage builds: Only copy the final artifact to the production stage.
- Combine RUN commands: Each RUN creates a layer. Chaining them reduces layers.
- Clean up in the same layer: Remove apt caches, temp files in the same RUN command.
- Use .dockerignore: Exclude node_modules, .git, tests from the build context.
- Avoid installing unnecessary packages: Use --no-install-recommends with apt-get.

Q13. What is Docker Compose and when would you use it?

▶️ Answer:

Docker Compose is a tool for defining and running multi-container Docker applications using a YAML file (docker-compose.yml). It manages the entire lifecycle — build, start, stop, scale — of all services in a single command.

Use it for: local development environments, running integration tests, simple multi-tier apps (web + db + cache).

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Q14. How do you handle secrets in Docker?

▶️ Answer:

Never put secrets in environment variables or bake them into images. Recommended approaches:

- Docker Secrets (Swarm): Store secrets in the Swarm's encrypted Raft store; mounted as files in /run/secrets/.
- Build-time: Use --secret flag with BuildKit to mount secrets during build without baking them in.
- External secrets managers: Vault, AWS Secrets Manager, Kubernetes Secrets (when running in K8s).
- Runtime injection: Pass secrets as environment variables from a secrets manager at startup (not hardcoded in Dockerfile).
- Avoid: ENV PASSWD=secret in Dockerfile — visible in docker history.

Q15. What happens when you run 'docker stop' vs 'docker kill'?

▶️ **Answer:**

docker stop: Sends a SIGTERM signal to the main process in the container, giving it time (default 10s) to shut down gracefully. If the process doesn't stop, it then sends SIGKILL.

docker kill: Immediately sends SIGKILL (or a signal you specify), forcefully terminating the container with no grace period.

Best practice: Use docker stop for graceful shutdowns. Use docker kill only when a container is unresponsive.

Q16. How do you debug a container that won't start?

▶️ **Answer:**

Debugging approach:

```
# Check logs from crashed container
docker logs <container_id>

# Inspect container state and config
docker inspect <container_id>

# Override entrypoint to get a shell
docker run -it --entrypoint sh myimage

# Check events
docker events
```

Q17. How does Docker use Linux namespaces and cgroups?

▶️ **Answer:**

Docker leverages core Linux kernel features to provide isolation and resource control:

Namespaces (isolation): Each container gets its own: PID namespace (isolated process tree), Network namespace (isolated network stack), Mount namespace (isolated filesystem), UTS namespace (isolated hostname), IPC namespace (isolated inter-process communication), User namespace (isolated user IDs).

cgroups (resource control): Control Groups limit and monitor how much CPU, memory, disk I/O, and network bandwidth a container can use. This prevents any single container from starving others.

☸️ KUBERNETES CORE CONCEPTS

Q18. What problem does Kubernetes solve?

▶️ Answer:

Kubernetes (K8s) is a container orchestration platform that solves the operational challenges of running containers at scale in production:

- Scheduling: Decides which node runs which container based on resource availability.
- Self-healing: Automatically restarts failed containers, replaces unhealthy nodes.
- Scaling: Scales applications up/down based on demand (HPA).
- Service discovery & load balancing: Assigns DNS names and distributes traffic across pods.
- Rolling deployments & rollbacks: Updates apps with zero downtime; rolls back on failure.
- Configuration management: Manages app config and secrets separately from code.
- Storage orchestration: Auto-mounts storage systems (local, cloud, NFS).

Q19. Explain the Kubernetes architecture.

▶️ Answer:

Kubernetes has a control plane and worker nodes:

- Control Plane: kube-apiserver (REST API gateway, single source of truth), etcd (distributed key-value store for all cluster state), kube-scheduler (places pods on nodes based on resource requirements/constraints), kube-controller-manager (runs controllers: node, replication, endpoints, etc.), cloud-controller-manager (interfaces with cloud providers).
- Worker Nodes: kubelet (agent that ensures containers in pods are running and healthy), kube-proxy (maintains network rules for pod communication), Container Runtime (Docker, containerd, CRI-O — runs the actual containers).

Q20. What are Pods, and why does Kubernetes use them instead of containers directly?

▶️ Answer:

A Pod is the smallest deployable unit in Kubernetes. It represents one or more containers that are tightly coupled, sharing the same network namespace (same IP, port space) and storage volumes.

Why pods (not bare containers):

- Sidecar pattern: Allows helper containers (log shippers, proxies) to run alongside the main app container.
- Shared networking: Containers in a pod communicate via localhost — no overhead.
- Atomic scheduling: Kubernetes schedules the entire pod as a unit, ensuring co-location.
- Lifecycle management: All containers in a pod start and stop together.

Q21. What is the difference between a Deployment, StatefulSet, and DaemonSet?

Answer:

Three different workload controllers for different use cases:

- Deployment: For stateless applications. Manages a ReplicaSet. Supports rolling updates and rollbacks. Pods are interchangeable — any pod can serve any request. Use for: web servers, APIs, microservices.
- StatefulSet: For stateful applications. Pods have stable, persistent hostnames (pod-0, pod-1) and dedicated PersistentVolumes. Ordered startup/shutdown. Use for: databases (MySQL, Cassandra, Kafka, Zookeeper).
- DaemonSet: Ensures one pod runs on every (or selected) node. New nodes get the pod automatically. Use for: log collectors (Fluentd), monitoring agents (Prometheus node-exporter), network plugins (Calico).

Q22. What is the role of etcd in Kubernetes?

Answer:

etcd is a distributed, consistent, highly-available key-value store that acts as the single source of truth for the entire Kubernetes cluster state. It stores all cluster data: pod specs, node info, config maps, secrets, service accounts, RBAC rules, etc.

The kube-apiserver is the only component that reads/writes to etcd directly. All other components communicate via the API server.

Critical importance: If etcd is lost without a backup, the entire cluster state is gone. In production, etcd should be run as a 3 or 5-node cluster for high availability.

Q23. What does kube-scheduler do?

Answer:

The kube-scheduler watches for newly created Pods that have no assigned node, and selects the best node for them to run on. It does this in two phases:

- Filtering (Predicates): Eliminates nodes that don't meet the pod's requirements — insufficient CPU/memory, taints, node affinity rules, port conflicts.
- Scoring (Priorities): Ranks the remaining nodes using scoring functions — prefers nodes with more free resources, spreading pods evenly, data locality, etc.
- The pod is then assigned to the highest-scored node via a binding operation.

KUBERNETES NETWORKING

Q24. How does pod-to-pod communication work in Kubernetes?

Answer:

The Kubernetes networking model requires: every pod gets a unique cluster-wide IP address, pods can communicate with any other pod without NAT, nodes can communicate with pods without NAT.

This is implemented by a CNI (Container Network Interface) plugin. Each pod's IP is routable within the cluster. When pod A talks to pod B's IP, the CNI plugin (Calico, Flannel, etc.) routes the traffic across nodes using techniques like VXLAN, BGP, or IPtables rules.

Q25. What is a Service? Explain ClusterIP, NodePort, and LoadBalancer.

▶️ Answer:

A Service provides a stable, abstract way to expose a set of pods. It gives a fixed IP/DNS name even as pods come and go. Types:

- ClusterIP (default): Exposes the service on a cluster-internal IP. Only accessible within the cluster. Used for internal microservice communication.
- NodePort: Exposes the service on a static port (30000-32767) on every node's IP. Accessible externally via <NodeIP>:<NodePort>. Useful for dev/testing.
- LoadBalancer: Provisions an external load balancer from the cloud provider (AWS ELB, GCP LB). The LB routes external traffic to the NodePort. Standard for production on cloud.
- ExternalName: Maps service to a DNS name. Used to alias external services inside the cluster.

Q26. What is an Ingress, and how does it differ from a Service?

▶️ Answer:

An Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services within a cluster. It provides path-based routing, host-based routing, SSL/TLS termination, and name-based virtual hosting — all in one place.

Key difference from Service: A LoadBalancer Service creates one cloud load balancer per service (expensive). Ingress uses a single load balancer with routing rules to direct traffic to many services — much more cost-effective.

An Ingress Controller (Nginx, Traefik, HAProxy, AWS ALB) is required to fulfill Ingress rules.

Q27. What is a CNI plugin? Name some examples.

▶️ Answer:

CNI (Container Network Interface) is a specification for network plugins that configure networking for pods. When a pod is created, Kubernetes calls the CNI plugin to set up the pod's network interface, IP address, and routes.

- Calico: Most popular for production. Uses BGP for routing. Supports Network Policies natively. High performance.
- Flannel: Simple overlay network using VXLAN. Easy to set up. No Network Policy support natively.
- Weave Net: Easy setup, automatic encryption, supports Network Policies.
- Cilium: eBPF-based. High performance, advanced observability, Layer 7 network policies.
- AWS VPC CNI: Assigns actual VPC IPs to pods on EKS. Full AWS networking integration.

💾 KUBERNETES STORAGE

Q28. What is a PersistentVolume (PV) vs PersistentVolumeClaim (PVC)?

▶️ Answer:

PersistentVolume (PV): A piece of storage in the cluster provisioned by an admin or dynamically by a StorageClass. It's a cluster resource like a node. It has a lifecycle independent of any pod.

PersistentVolumeClaim (PVC): A request for storage by a user/pod. Similar to how a pod consumes node resources, a PVC consumes PV resources. The user specifies size and access mode; Kubernetes binds it to a matching PV.

Analogy: PV is the actual storage disk; PVC is the request/ticket to use that disk.

Q29. What is a StorageClass?

▶️ Answer:

A StorageClass describes the 'class' of storage — the provisioner, reclaim policy, and parameters. It enables dynamic provisioning: when a PVC requests a StorageClass, Kubernetes automatically creates a matching PV.

- Examples: standard (GKE), gp2/gp3 (EKS/AWS EBS), managed-premium (AKS), local-path.
- Reclaim policies: Retain (keep data after PVC deleted), Delete (auto-delete underlying storage), Recycle (deprecated).
- Without StorageClass: Admin must manually pre-provision PVs.

Q30. What is the difference between ReadWriteOnce, ReadOnlyMany, and ReadWriteMany?

▶️ Answer:

These are PersistentVolume access modes, defining how the volume can be mounted:

- ReadWriteOnce (RWO): Volume can be mounted as read-write by a single node. Most block storage (EBS, GCE PD) supports this.
- ReadOnlyMany (ROX): Volume can be mounted as read-only by many nodes simultaneously. Useful for shared configuration/data.
- ReadWriteMany (RWX): Volume can be mounted as read-write by many nodes simultaneously. Requires shared file systems like NFS, EFS, CephFS, or Azure Files.
- ReadWriteOncePod (RWOP): Kubernetes 1.22+. Only one pod cluster-wide can mount read-write.

🔧 KUBERNETES CONFIGURATION & SECRETS

Q31. What is a ConfigMap, and when would you use it?

▶️ Answer:

A ConfigMap is a Kubernetes API object that stores non-confidential configuration data as key-value pairs. It decouples configuration from application code, enabling the same container image to be used in different environments (dev/staging/prod) with different configs.

Usage: Database hostnames, feature flags, environment names, configuration files.

Injection methods: Environment variables, command-line arguments, files in a volume mount.

Q32. How are Kubernetes Secrets stored and what are their limitations?

▶️ Answer:

Secrets store sensitive data (passwords, tokens, TLS certs) as base64-encoded strings in etcd. They can be injected as env vars or volume-mounted files.

Critical limitations:

- Base64 is NOT encryption — it's just encoding. Anyone with API access can decode it.
- By default, Secrets in etcd are NOT encrypted at rest (requires Encryption at Rest configuration).
- Secrets are scoped to a namespace.
- Best practices: Enable etcd encryption at rest, use RBAC to restrict Secret access, consider external secret stores (HashiCorp Vault, AWS Secrets Manager) with the External Secrets Operator or Sealed Secrets.

📈 KUBERNETES SCALING & SCHEDULING

Q33. What is the Horizontal Pod Autoscaler (HPA)?

▶️ Answer:

The HPA automatically scales the number of pod replicas in a Deployment/StatefulSet/ReplicaSet based on observed metrics. By default it uses CPU utilization (from metrics-server), but can also use custom metrics (via Prometheus Adapter) or external metrics.

How it works: The HPA controller checks metrics every 15 seconds. It calculates desired replicas = ceil(current replicas * (current metric / target metric)), then scales within min/max replica bounds.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Q34. What are resource requests vs. limits, and why do they matter?

▶️ Answer:

Requests and limits are set per container in the pod spec:

- requests: The amount of resources guaranteed to the container. Used by the scheduler to decide which node can fit the pod.
- limits: The maximum resources the container can use. If CPU exceeds the limit, it's throttled. If memory exceeds the limit, the container is OOMKilled.
- QoS Classes: Guaranteed (requests == limits), Burstable (requests < limits), BestEffort (no requests/limits set).
- Best practice: Always set both. Under-setting causes OOMKills; over-setting wastes cluster resources.

Q35. What are taints, tolerations, and node affinity?

▶️ Answer:

These are mechanisms to control pod placement on nodes:

- Taints: Applied to nodes to repel pods. A taint has a key, value, and effect (NoSchedule, PreferNoSchedule, NoExecute).
- Tolerations: Applied to pods to allow them to be scheduled on tainted nodes. A pod must tolerate a taint to run on that node.
- Node Affinity: Rules on pods specifying which nodes they prefer or require, based on node labels. More expressive than nodeSelector.
- Use case: Dedicate GPU nodes — taint GPU nodes, only GPU workloads tolerate the taint. Or spread workloads across zones using affinity rules.

Q36. What is a Pod Disruption Budget (PDB)?

▶️ Answer:

A PDB limits the number of pods of a replicated application that can be down simultaneously during voluntary disruptions (node drains for maintenance, cluster upgrades).

minAvailable: At least this many pods must remain available. maxUnavailable: At most this many pods can be unavailable.

PDBs protect against voluntary disruptions only — not hardware failures or out-of-memory kills. Essential for running zero-downtime operations in production.

🚀 KUBERNETES DEPLOYMENTS & DEBUGGING

Q37. How do you perform a rolling update? How do you roll back?

▶️ Answer:

Kubernetes Deployments support rolling updates natively. When you update a Deployment (e.g., new image version), it gradually replaces old pods with new ones based on the strategy:

```
strategy:
  type: RollingUpdate
```

```
  rollingUpdate:
    maxUnavailable: 1   # Max pods down at once
    maxSurge: 1         # Max extra pods during update

# Update the image:
kubectl set image deployment/my-app app=myimage:v2

# Check rollout status:
kubectl rollout status deployment/my-app

# Rollback to previous version:
kubectl rollout undo deployment/my-app

# Rollback to specific revision:
kubectl rollout undo deployment/my-app --to-revision=3
```

Q38. What is a liveness probe vs. readiness probe vs. startup probe?

▶️ Answer:

Kubernetes uses probes to check container health:

- Liveness Probe: Checks if the container is alive. If it fails, Kubernetes restarts the container. Use for detecting deadlocks where the app is running but unable to make progress.
- Readiness Probe: Checks if the container is ready to receive traffic. If it fails, the pod is removed from the Service's load balancer (traffic stops). The container is NOT restarted. Use for apps that need time to warm up (load DB connections, caches).
- Startup Probe: Checks if the application has started. Disables liveness/readiness probes until it succeeds. Use for slow-starting containers to prevent them being killed before they're ready.
- Probe types: HTTP GET, TCP Socket, exec (run a command).

Q39. What happens when a pod goes into CrashLoopBackOff?

▶️ Answer:

CrashLoopBackOff means the container is crashing repeatedly and Kubernetes keeps restarting it with an exponential back-off delay (10s → 20s → 40s → max 5 minutes).

Common causes: Application bug/exception at startup, misconfigured environment variables/secrets, wrong command/entrypoint, liveness probe misconfigured, OOMKilled (out of memory).

```
# Check pod status and events
kubectl describe pod <pod-name>

# Check current logs
kubectl logs <pod-name>

# Check previous container logs
kubectl logs <pod-name> --previous
```

```
# Check if OOMKilled
kubectl get pod <pod-name> -o
jsonpath='{.status.containerStatuses[0].lastState}'
```

Q40. How do you debug a pod stuck in Pending state?

▶️ Answer:

A Pending pod means the scheduler cannot place it on any node. Check with 'kubectl describe pod <name>' and look at Events.

Common causes:

- Insufficient resources: No node has enough CPU/memory. Fix: scale the cluster or reduce requests.
- No matching nodes: Node affinity, nodeSelector, or taints/tolerations prevent scheduling.
- PVC not bound: The pod requires a PVC that isn't bound to a PV. Check PVC status.
- Image pull issues: Shown as ErrImagePull or ImagePullBackOff, not Pending.
- Namespace resource quotas: The namespace has hit its quota limit.

🔒 KUBERNETES SECURITY

Q41. What is RBAC? Explain Roles, ClusterRoles, and RoleBindings.

▶️ Answer:

RBAC (Role-Based Access Control) is the authorization mechanism in Kubernetes that regulates access to API resources based on the roles assigned to users/service accounts.

- Role: Defines permissions (verbs: get, list, create, delete) on resources (pods, services) within a specific namespace.
- ClusterRole: Like Role, but cluster-wide. Can grant access across all namespaces or to cluster-scoped resources (nodes, PVs).
- RoleBinding: Grants a Role (or ClusterRole) to a user/group/serviceaccount within a namespace.
- ClusterRoleBinding: Grants a ClusterRole cluster-wide.
- Best practice: Follow principle of least privilege — grant only the minimum permissions needed.

Q42. What is a ServiceAccount?

▶️ Answer:

A ServiceAccount provides an identity for processes running in Pods to authenticate with the Kubernetes API. Every pod runs under a ServiceAccount (default if not specified).

When a pod uses a ServiceAccount, a JWT token is automatically mounted at /var/run/secrets/kubernetes.io/serviceaccount/token. This token is used to authenticate API requests.

Best practice: Create dedicated ServiceAccounts per application and bind only necessary RBAC permissions. Disable auto-mounting for pods that don't need API access with automountServiceAccountToken: false.

Q43. How do you manage secrets securely in Kubernetes?

▶️ Answer:

Defense-in-depth approach to secret management:

- Enable Encryption at Rest: Configure the kube-apiserver to encrypt Secret data in etcd using AES-CBC or AES-GCM.
- Restrict RBAC: Use least privilege — not everyone should be able to read Secrets.
- External Secrets Operator: Sync secrets from AWS Secrets Manager, GCP Secret Manager, or HashiCorp Vault into Kubernetes Secrets automatically.
- Sealed Secrets (Bitnami): Encrypt secrets into SealedSecret objects that are safe to commit to Git. Only the cluster can decrypt them.
- Vault Agent Injector: HashiCorp Vault injects secrets directly into pod filesystems without storing them in Kubernetes Secrets at all.

⚡ KUBERNETES ADVANCED TOPICS

Q44. What is a Helm chart and what problem does Helm solve?

▶️ Answer:

Helm is the package manager for Kubernetes. A Helm chart is a collection of files that describe a related set of Kubernetes resources. Charts can be versioned, shared, and reused.

Problems Helm solves:

- Templating: Avoids duplicating Kubernetes YAML. Variables make configs reusable across environments.
- Dependency management: Charts can depend on other charts (e.g., app chart depends on postgresql chart).
- Release management: Helm tracks each deployment as a 'release', enabling easy upgrades and rollbacks (helm upgrade, helm rollback).
- Distribution: Charts are packaged and distributed via chart repositories (Artifact Hub, private registries).

Q45. What are Operators and Custom Resource Definitions (CRDs)?

▶️ Answer:

CRDs (Custom Resource Definitions) allow you to extend the Kubernetes API with custom resource types. You define a new API object (e.g., 'Database', 'KafkaCluster') and Kubernetes accepts and stores them like any built-in resource.

Operators are applications that use CRDs + custom controllers to manage complex, stateful applications. They encode operational knowledge (install, upgrade, backup, failover) into software.

Examples: prometheus-operator, cert-manager, strimzi (Kafka), mongodb-community-operator, zalando postgres-operator.

Q46. What is a service mesh (e.g., Istio, Linkerd)?

▶️ Answer:

A service mesh is a dedicated infrastructure layer for handling service-to-service communication in a microservices architecture. It's typically implemented using sidecar proxies (Envoy for Istio) injected into each pod.

Features provided by a service mesh:

- mTLS: Automatic mutual TLS encryption between all services.
- Traffic management: Fine-grained routing, A/B testing, canary deployments, traffic shifting.
- Observability: Distributed tracing, metrics, service topology maps (with zero code changes).
- Resilience: Circuit breaking, retries, timeouts, rate limiting.
- When to use: Large microservices architectures (10+ services) where cross-cutting concerns become complex to manage in application code.

Q47. How would you achieve zero-downtime deployments in Kubernetes?

▶️ Answer:

Zero-downtime deployment checklist:

- Use RollingUpdate strategy with maxUnavailable: 0 so old pods remain until new ones are ready.
- Configure readiness probes so new pods receive traffic only when truly ready.
- Set preStop lifecycle hooks and terminationGracePeriodSeconds to allow in-flight requests to complete.
- Use Pod Disruption Budgets to protect against voluntary disruptions.
- Implement graceful shutdown in your application — handle SIGTERM, finish current requests, close DB connections.
- Use multiple replicas so traffic is never completely interrupted during rollout.

Q48. How does Kubernetes handle node failure?

▶️ Answer:

When a node fails, the process unfolds in stages:

- Detection: The node controller monitors node heartbeats. After 40 seconds of no heartbeat, node is marked NotReady. After 5 minutes (default), it's considered unreachable.
- Pod eviction: Pods on the failed node are marked for deletion and rescheduled on healthy nodes (if controlled by a Deployment/ReplicaSet).
- StatefulSet pods: Require manual intervention if the node is truly gone (to prevent split-brain scenarios).
- Non-managed pods (standalone): Are NOT rescheduled automatically — another reason to always use controllers like Deployments.
- PodDisruptionBudget: Protects against accidental disruption but cannot protect against node failures.

🎯 SCENARIO-BASED QUESTIONS

Q49. How would you investigate a pod that keeps restarting?

▶️ **Answer:**

Systematic debugging approach:

```
# Step 1: Check pod status and restart count
kubectl get pod <name> -o wide

# Step 2: Look at events and last exit code
kubectl describe pod <name>

# Step 3: Check current logs
kubectl logs <name>

# Step 4: Check logs from previous crashed container
kubectl logs <name> --previous

# Step 5: Check if OOMKilled (exit code 137)
kubectl get pod <name> -o
jsonpath='{.status.containerStatuses[0].lastState.terminated}'

# Step 6: If unclear, exec into a running instance
kubectl exec -it <name> -- /bin/sh
```

Q50. Traffic is not reaching your pod. How do you troubleshoot?

▶️ **Answer:**

Work top-down through the networking stack:

- Check pod is Running and Ready: kubectl get pods — if not Ready, readiness probe is failing.
- Check Service selector matches pod labels: kubectl describe svc <name> — Endpoints should list pod IPs.
- Check Service has endpoints: kubectl get endpoints <name> — empty means label mismatch.
- Test internal connectivity: kubectl exec -it <pod> -- curl http://<service-name>:<port>
- Check Ingress configuration: kubectl describe ingress <name> — verify rules and backend service.
- Check Network Policies: Ensure no NetworkPolicy is blocking traffic.
- Check application logs: Is the app actually listening on the expected port?

Q51. How would you reduce cloud costs in a Kubernetes cluster?

▶️ **Answer:**

Cost optimization strategies:

- Right-size resources: Set accurate requests/limits. Use VPA recommendations. Avoid over-provisioning.
- Cluster Autoscaler: Scale nodes down when underutilized, up when pods are pending.
- Spot/Preemptible nodes: Use spot instances for fault-tolerant workloads (70-90% cheaper).
- HPA: Scale down to minimum replicas during off-peak hours.
- Namespace resource quotas: Prevent teams from accidentally requesting excessive resources.
- Karpenter (AWS): More efficient node provisioning — provisions exact instance sizes needed.
- Delete unused resources: Clean up old deployments, services, PVCs, images in ECR/GCR.

💡 Interview Tips

• Know your kubectl commands: get, describe, logs, exec, apply, rollout
• Be able to draw and explain both Docker and Kubernetes architectures
• Discuss trade-offs — interviewers love 'it depends' answers backed by reasoning
• Always relate answers to real production scenarios and lessons learned